UCS 503P — Software Engineering Project

UniVerse: A Skill Swap Community Platform

Peer-to-Peer Teaching Platform with Real-Time Chat and Credit-Based Incentives

Submitted by:

1024030215 Sehajneet Kaur
1024030711 Raynee Jindal
1024030247 Sahejbir Singh
1024030649 Shaurya Ranjan

Group: 3C52 BE Third Year, COE

Submitted to:

Paramveer Kaur Ma'am

Project Report Prototype Stage

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1 Introduction **3**
- 1.1 Background and Context . . . 3
  - 1.1.1 Problem Statement . . . 3
- 1.2 Project Objectives . . . 3
- 1.3 Project Scope . . . 4

2 Methodology and Design **5**
- 2.1 System Architecture . . . 5
  - 2.1.1 High-Level Design . . . 5
  - 2.1.2 Activity Diagram . . . 6
  - 2.1.3 Class Diagram . . . 6
  - 2.1.4 Technical Stack . . . 6
- 2.2 Implementation Details . . . 6
  - 2.2.1 Backend Modules . . . 6
  - 2.2.2 Database Schema Highlights . . . 9
  - 2.2.3 Security Measures . . . 9
- 2.3 Frontend Architecture . . . 9

3 Results and Evaluation **10**
- 3.1 Development Milestones . . . 10
- 3.2 Testing Strategy . . . 10
  - 3.2.1 Unit Testing . . . 10
  - 3.2.2 Integration Testing . . . 10
  - 3.2.3 Security Testing . . . 10
- 3.3 Challenges Encountered and Resolution . . . 11
  - 3.3.1 Challenge 1: Cloudinary Image Cleanup . . . 11
  - 3.3.2 Challenge 2: Real-Time Chat with Persistence . . . 11
  - 3.3.3 Challenge 3: Closed Post Enforcement . . . 11

4 Conclusion **12**
- 4.1 Summary of Work . . . 12
- 4.2 Key Findings . . . 12
- 4.3 Future Work and Recommendations . . . 12
- 4.4 Learning Outcomes . . . 13
- 4.5 Final Remarks . . . 13

Bibliography **14**

Chapter 1

Introduction

1.1 Background and Context

UniVerse is a web-based peer-to-peer skill exchange platform designed exclusively for the Thapar Institute of Engineering and Technology (TIET) community. Students possess diverse skills but lack a structured, incentivized way to teach and learn from one another. The motivation behind UniVerse stems from three observations:

1. **Unutilized Peer Expertise:** Students often struggle to find classmates with specific skills.
2. **Resource Inefficiency:** Books and lab equipment are borrowed informally or through inefficient channels.
3. **Community Isolation:** The campus lacks a unified channel for lost-and-found, discussions, and resource exchange.

1.1.1 Problem Statement

Students at TIET require a platform to:

- Discover and request peer-taught sessions, and offer their own skills to peers.
- Coordinate learning through real-time chat.
- Borrow, lend, and report lost items across the community.

1.2 Project Objectives

1. **Skill swap subsystem:** Enable students to offer and request peer-taught sessions with real-time negotiation via chat.
2. **Credit-based incentive system:** Reward teaching (+10 credits per session) and deduct for learning (-5 credits per session) with atomic correctness.
3. **Community and admin features:** Support borrow/lend posts, lost-and-found reports, discussions, and moderation tools.
4. **Security:** Restrict access to @thapar.edu accounts and enforce role-based authorization.

1.3 Project Scope

- **User Base:** TIET students with valid @thapar.edu Google accounts only.
- **Core Features:** Skill Swap, Utility Channels (General, Borrow/Lend, Lost & Found), real-time chat, notifications, and admin management.

Chapter 2

Methodology and Design

The project followed an iterative, feature-by-feature development approach.

2.1 System Architecture

UniVerse is a three-tier web application:

1. **Frontend:** React 18 + Vite, communicating with the backend via REST API and WebSocket.
2. **Backend:** Node.js + Express 5, implementing business logic, authentication, authorization, and real-time messaging.
3. **Data Layer:** MongoDB Atlas, storing user profiles, skills, sessions, posts, messages, and transactions.

2.1.1 High-Level Design

Authentication Flow

- Google OAuth 2.0 is the sole authentication mechanism, restricted to the @thapar.edu domain.
- A short-lived JWT access token and a refresh token are issued as HTTP-only cookies.

Credit System

- New users start with 50 credits; a completed session gives the mentor +10 and charges the learner -5.
- Credit updates use MongoDB atomic operations, and session completion is idempotent.
- Admins can deduct credits with a mandatory reason (e.g., no-show penalty).

Real-Time Messaging

- Socket.IO enables one-to-one chat between session participants, authenticated with JWT.
- All messages are persisted to MongoDB so history survives disconnections.

Community Channels

- Three fixed channels: General, Borrow/Lend, and Lost & Found.
- Posts in Borrow/Lend and Lost & Found can be closed, after which no interactions are allowed.
- General supports upvotes/downvotes, nested comments, saving, and reporting.

2.1.2 Activity Diagram

Figure 2.1 shows the skill swap workflow, from listing a skill to rating the session.

2.1.3 Class Diagram

Figure 2.2 shows the UML class diagram of the main entities, their key attributes, public methods, and relationships. Enumeration-typed fields (e.g., `SessionStatus`, `ChannelType`) are shown by their type name.

2.1.4 Technical Stack

- **Frontend:** React 18, Vite, React Query, Axios, Socket.IO Client.
- **Backend:** Node.js, Express 5, Mongoose, Socket.IO.
- **Database and Services:** MongoDB Atlas, Google OAuth 2.0, Cloudinary (image storage).

2.2 Implementation Details

2.2.1 Backend Modules

- **Authentication:** Google OAuth callback with domain validation, JWT generation, and profile/avatar updates.
- **Sessions and Credits:** Session lifecycle (pending → accepted → scheduled → completed) with atomic credit updates and transaction logging.
- **Posts and Comments:** Post creation with images, nested comments, and closed-post enforcement.
- **Admin:** User suspension/banning, content removal, report review, and credit deduction.

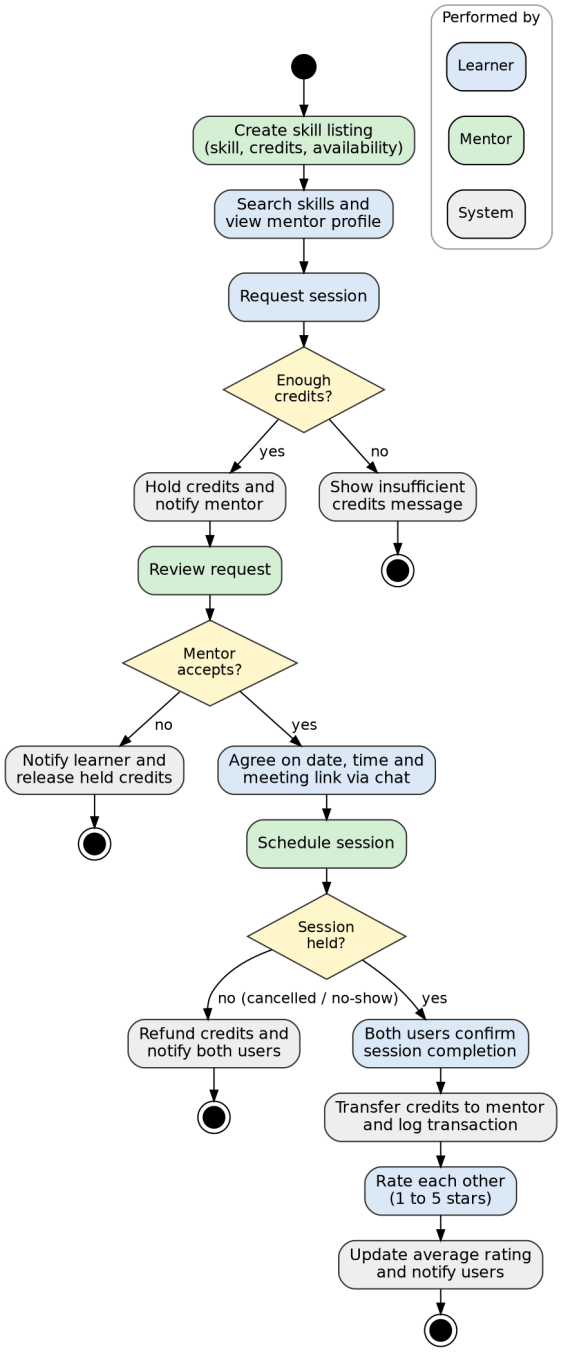


Figure 2.1: Skill Swap Activity Diagram

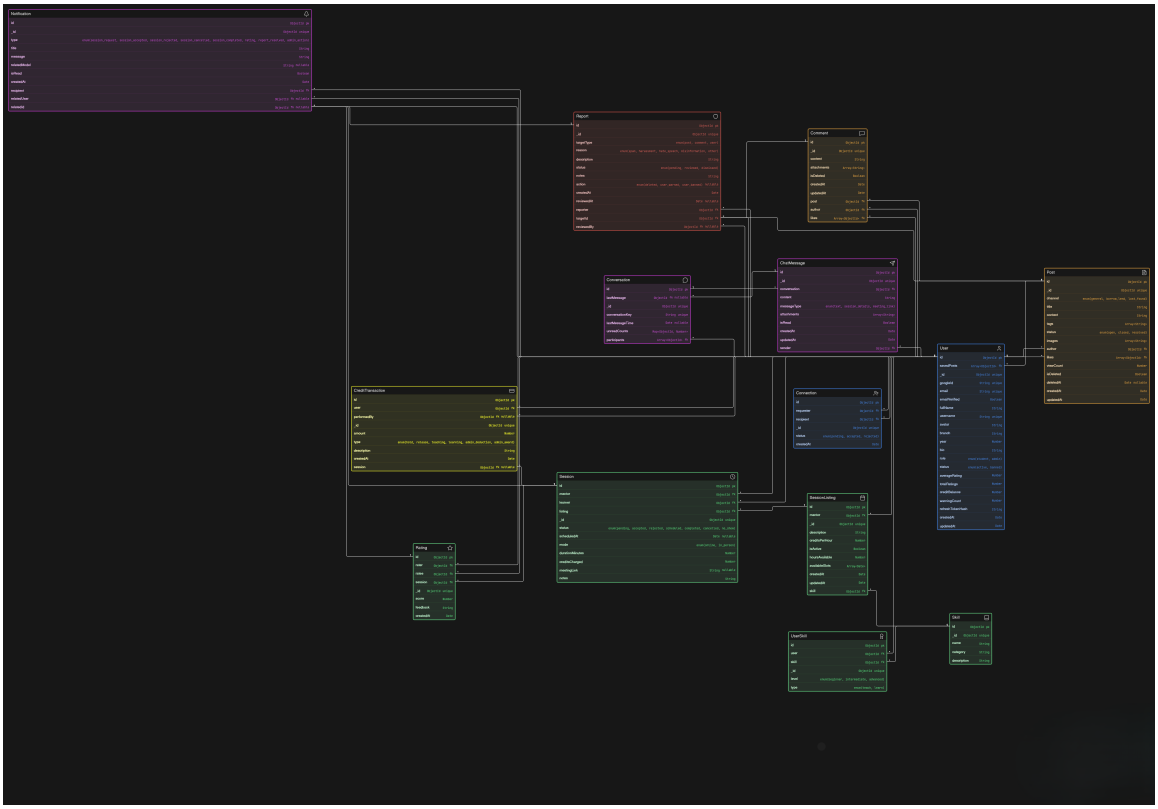


Figure 2.2: UniVerse UML Class Diagram

2.2.2 Database Schema Highlights

- **User:** profile details, credits (default 50, minimum 0), role, status, and skills to teach/learn.
- **Session:** mentor, learner, skill, status, mode, date, and meeting link.
- **CreditTransaction:** user, amount, type, and session reference for an audit trail.

2.2.3 Security Measures

1. **Google OAuth:** Only @thapar.edu accounts can sign in; tokens are stored in HTTP-only cookies.
2. **Admin Authorization:** Role-based access control ensures only admins can access admin endpoints.
3. **Rate Limiting:** Auth and general API endpoints are rate-limited to mitigate brute-force attacks.

2.3 Frontend Architecture

- **State Management:** Context API with `useReducer` for authentication, and React Query for server state.
- **Real-Time Chat:** Socket.IO client connection authenticated with JWT.
- **Routing:** React Router v6 with protected routes for authenticated pages.

Chapter 3

Results and Evaluation

3.1 Development Milestones

Milestone	Status
Backend API	Completed
MongoDB Schema	Completed
Authentication	Completed
Credit System	Completed
Frontend Components	Completed
Real-Time Chat (Socket.IO)	Completed
Admin Features	Completed

Table 3.1: Development Milestones and Status

3.2 Testing Strategy

3.2.1 Unit Testing

- Credit calculation logic (teach +10, learn -5, floor at 0).
- Session status transitions and authorization checks.

3.2.2 Integration Testing

- End-to-end authentication flow (Google OAuth, token refresh, logout).
- Session completion with credit transfer.
- Real-time chat message persistence and retrieval.

3.2.3 Security Testing

- Non-Thapar accounts are rejected at the OAuth callback.
- Non-admin users cannot access admin endpoints.
- Credits cannot go negative.

3.3 Challenges Encountered and Resolution

3.3.1 Challenge 1: Cloudinary Image Cleanup

Problem: When users delete posts or upload new avatars, orphaned images remain in Cloudinary, consuming storage quota.

Solution: Implemented cleanup routines in the post deletion and avatar update controllers that delete the old image through Cloudinary's API.

Result: No orphaned images remained after testing.

3.3.2 Challenge 2: Real-Time Chat with Persistence

Problem: Socket.IO messages are ephemeral; if a user disconnects, message history is lost.

Solution: All messages are persisted to MongoDB with a compound index on (`conversationId, createdAt`). On reconnection, the frontend fetches recent history from the API.

Result: Chat history survives page reloads and network disconnections.

3.3.3 Challenge 3: Closed Post Enforcement

Problem: Frontend-only enforcement can be bypassed by direct API calls.

Solution: Enforcement was moved to the backend controllers, which return a 409 Conflict response when a user interacts with a closed post.

Result: No interactions are possible on closed posts, even via direct API calls.

Chapter 4

Conclusion

4.1 Summary of Work

UniVerse realizes a full-stack peer-to-peer skill exchange platform tailored for the TIET community. The team delivered:

1. A backend API supporting authentication, skill discovery, session management, credit transactions, community channels, and admin tooling.
2. A credit-based economy that incentivizes teaching while keeping learning accessible.
3. Real-time chat with persistent message history for coordinating sessions.

4.2 Key Findings

- **Atomic Operations are Critical:** The credit system's correctness depends on atomic database operations.
- **Backend-First Security:** Business rules must be enforced on the backend; frontend validation is only for user experience.
- **Persistence for Real-Time Features:** Pairing Socket.IO with MongoDB lets chat history survive disconnections.

4.3 Future Work and Recommendations

1. **Deployment:** The platform is not yet deployed. Host the frontend, backend, and database on production infrastructure so it is available to the TIET community.
2. **User Search:** Add user search based on rating, which is not implemented yet.
3. **Skill Proficiency Levels:** Extend the Skill model with proficiency levels (Beginner, Intermediate, Advanced) for more granular matching.
4. **Recommendation Engine:** Build a machine learning model to recommend skills to learn based on user history and peer trends.

4.4 Learning Outcomes

- **Distributed Systems:** Race conditions, atomicity, and idempotency.
- **Security:** OAuth flows, JWT token management, and role-based authorization.
- **Real-Time Communication:** WebSocket fundamentals and persistence strategies.

4.5 Final Remarks

UniVerse shows that a peer-to-peer community platform can be built with modern technologies. The combination of a credit system, real-time communication, and community channels gives students a reason to participate and invest in their peers' growth. With deployment and the planned enhancements, it can become a useful platform for informal learning at TIET.

Bibliography

[Aut24] Auth0. *Introduction to JSON Web Tokens (JWT)*. 2024. URL: https://jwt.io/.

[Clo24a] Google Cloud. *Using OAuth 2.0 to Access Google APIs*. 2024. URL: https://developers.google.com/identity/protocols/oauth2.

[Clo24b] Cloudinary. *Cloudinary: Image and Video Cloud Platform*. 2024. URL: https://cloudinary.com/.

[Con24a] Expressjs Contributors. *Express: Fast, unopinionated, minimalist web framework for Node.js*. 2024. URL: https://expressjs.com/.

[Con24b] Helmet Contributors. *Helmet: Help secure Express apps with various HTTP headers*. 2024. URL: https://helmetjs.github.io/.

[Con24c] Mongoose Contributors. *Mongoose: Elegant MongoDB object modeling*. 2024. URL: https://mongoosejs.com/.

[Con24d] Socket.IO Contributors. *Socket.IO: Enables real-time bidirectional and event-based communication*. 2024. URL: https://socket.io/.

[Inc24a] MongoDB Inc. *MongoDB Atlas: Cloud Database Service*. 2024. URL: https://www.mongodb.com/cloud/atlas.

[Inc24b] MongoDB Inc. *MongoDB: The Most Popular NoSQL Database*. 2024. URL: https://www.mongodb.com/.

[Inc24c] Postman Inc. *Postman: API Collaboration Platform*. 2024. URL: https://www.postman.com/.

[KJSR26] Sehajneet Kaur, Raynee Jindal, Sahejbir Singh, and Shaurya Ranjan. *UniVerse: A Skill Swap Community Platform*. Source code repository, https://github.com/Saejdot23/ucs503p-202627-UniVerse. 2026.

[PM24] Niels Provos and David Mazières. *bcrypt: A password hashing library*. 2024. URL: https://www.npmjs.com/package/bcrypt.